

AutoLISP Runtime

User Manual

clautolisp contributors

May 25, 2026

Contents

1	Introduction	1
2	Loading	1
3	Running AutoLISP code	1
4	Building a context manually	2
5	Variable / function binding	2
6	The HAL backend	3
7	Encoding	3
8	Error handling	3
9	Printing AutoLISP values	3
10	See also	4
11	Synchronisation	4

1 Introduction

This manual is the programmer’s guide to the `clautolisp.autolisp-runtime` system. The audience is a Common Lisp developer integrating the runtime — embedding the AutoLISP evaluator in a host application, adding new builtins, or driving it from CL code for tests or batch processing.

The CLI front-ends (`clautolisp(1)` and `alfe(1)`) wrap this system; their man pages cover end-user concerns. Library mechanics live here.

2 Loading

The system is named `clautolisp/autolisp-runtime` in `clautolisp.asd`. Load it via ASDF:

```
(asdf:load-system "clautolisp/autolisp-runtime")
```

The dependency is `clautolisp/autolisp-reader` and `uiop`.

3 Running AutoLISP code

The high-level entry points are:

- `run-autolisp-string` TEXT &key dialect setup-fn — parse TEXT, run every form in a fresh runtime session, return the value of the last form.
- `run-autolisp-file` PATH &key dialect — load and run the file's forms in a fresh session.
- `autolisp-eval` FORM &optional context — evaluate a single pre-parsed runtime value against the given evaluation context.
- `autolisp-eval-progn` FORMS context — evaluate a sequence, returning the last value.

The `:dialect` keyword takes a dialect descriptor from `clautolisp.autolisp-reader` (`autolisp-dialect-strict`, `autolisp-dialect-autocad-2026`, `autolisp-dialect-bricscad-v26`).

4 Building a context manually

For long-lived sessions, build a context once and reuse it:

```
(let* ((context (make-default-runtime-context :dialect (autolisp-dialect-strict))))  
  ;; install core builtins (separate system) (clautolisp.autolisp-builtins-core:install-core-builtins) ;; run a few forms (autolisp-eval-progn forms context))
```

The context owns a session; the session owns a document namespace. Inspect via `evaluation-context-session`, `runtime-session-current-document`, etc.

5 Variable / function binding

The runtime is Lisp-1: SETQ and DEFUN write the same per-symbol binding cell. Read and write via:

- `lookup-variable` SYMBOL &optional CONTEXT → (values value boundp origin)
- `lookup-function` SYMBOL &optional CONTEXT → (values function boundp origin)
- `set-variable` SYMBOL VALUE &optional CONTEXT
- `set-function` SYMBOL FUNCTION &optional CONTEXT
- `intern-autolisp-symbol` NAME → autolisp-symbol

Both lookup functions walk the dynamic-frame stack first, then fall back to the namespace cell. `set-variable` mirrors: updates the innermost dynamic shadow if one exists, else writes to the namespace cell.

6 The HAL backend

When the runtime is asked to perform a host-side AutoLISP operation (`ent-get`, `ssget`, `getvar`, `command`, ...) it delegates to the session's HAL backend. Install one via:

```
(set-runtime-session-host session host-instance)
```

See `clautolisp.autolisp-host` for the protocol and `clautolisp.autolisp-mock-host` for the in-memory test backend. When no HAL is attached, host-side ops signal `:host-not-supported`.

7 Encoding

The session's `default-source-encoding` slot lets code override the dialect's default for file loads:

```
(set-default-source-encoding context :utf-8)
```

The CLI's `-e ENC` flag wires this on startup; programmatic code can call `set-default-source-encoding` at any point.

8 Error handling

Runtime errors signal `autolisp-runtime-error` with a structured `:code` and `:message`. The CL handler chain is the natural way to catch them:

```
(handler-case (autolisp-eval form context) (autolisp-runtime-error (condition) (format t "AutoLISP error: ~A~%" (autolisp-runtime-error-message condition))))
```

The `call-with-autolisp-error-handler` helper wraps a thunk with the same handler the CLI uses, surfacing the call stack on debug-mode invocations.

9 Printing AutoLISP values

The runtime installs CL `print-object` methods so an `autolisp-string` prints as `abc` or `"abc"` (depending on `*print-escape*`), an `autolisp-symbol` prints as its name, an `autolisp-ename` prints as `<Entity name: ...>`, and so on. The print pathway honours `clautolisp.autolisp-runtime:*color-output*` when set to a keyword accent (`:yellow` / `:blue`) — symbols get wrapped in the matching ANSI sequence.

10 See also

- autolisp-runtime-specifications.org — internal design + invariants.
- autolisp-spec/documentation/autolisp-visual-lisp-specification-draft.org — the normative AutoLISP language reference.
- `clautolisp.autolisp-builtins-core` — the standard builtin set this runtime expects loaded for real AutoLISP code to run.
- `clautolisp.autolisp-host` — HAL contract.

11 Synchronisation

Per `AGENTS.md`, code / spec / user-manual changes propagate.